Tyler Buck
CS499
July 24th, 2026

Artifact Description

The artifact selected for my Algorithms and Data Structures enhancement is the ABCU Advising Assistance Program, originally created for CS 300 in October 2025. The program is a C++ console application designed to assist with academic course planning. It loads course information from a CSV file, stores the course records in an unordered_map, sorts course identifiers for display, and allows a user to search for an individual course and view its prerequisites.

I selected this artifact because it already demonstrates several important concepts related to algorithms and data structures. The original program uses an unordered_map to associate course identifiers with Course objects and uses std::sort to display the available courses in alphanumeric order. These choices allow the program to provide efficient course lookup while still presenting the complete course catalog in an organized format.

Justification for Inclusion and Enhancement

I chose this artifact because it demonstrates practical use of data structures, searching, sorting, and related course relationships. After reviewing the original code, I found that the unordered_map and std::sort were already appropriate choices, so I focused on improving data validation rather than replacing working functionality.

The enhanced version adds three improvements. It rejects course records with missing IDs or titles, detects duplicate course IDs before insertion, and validates prerequisite references after the catalog is loaded. These changes improve data integrity and reduce the chance of invalid or inconsistent records being used by the application.

The duplicate check continues to use the existing unordered_map, so course-ID lookup remains efficient. The prerequisite validation adds a separate pass through the course

relationships, creating a reasonable trade-off between slightly more processing during loading and improved reliability.

Course Outcome Alignment

This enhancement most directly demonstrates Course Outcome 3, which focuses on designing and evaluating computing solutions using algorithmic principles and appropriate computer science practices while managing the trade-offs involved in design decisions.

One of the most important decisions I made during this enhancement was not to replace the existing data structure simply because the artifact was being used for the Algorithms and Data Structures category. After reviewing the original implementation, I found that unordered_map already provided an appropriate solution for storing and retrieving courses by their identifiers. Instead of introducing unnecessary complexity, I expanded the algorithms surrounding that structure to improve validation and data integrity.

The enhancement also demonstrates Course Outcome 4 through the practical use of C++ programming techniques, standard library containers, searching, sorting, and validation logic to improve the reliability of the application. Rather than changing the project only for appearance, the enhancement addresses specific weaknesses that could affect the accuracy of the course catalog.

The implementation remained consistent with the overall intent of my original enhancement plan, but the specific approach became more focused after reviewing the original code in greater detail. Since unordered_map and std::sort functionality were already implemented, I shifted the enhancement toward validation, duplicate detection, and prerequisite verification. This allowed me to make a more meaningful improvement without claiming functionality that was already present in the original artifact.

Algorithm and Data Structure Analysis

- The existing unordered_map supports average O(1) course lookup, which is also used for duplicate detection. The prerequisite validation checks each stored prerequisite against the course map, giving the validation pass an average complexity of approximately O(n + p), where n is the number of courses and p is the total number of prerequisite references.
- The existing course-list sorting remains O(n log n) through std::sort.
- The enhancement does not make every operation faster. Instead, it adds validation during loading to improve the reliability of the course data while preserving efficient lookup behavior.

Reflection on the Enhancement Process

The main lesson from this enhancement was that improving an application does not always require replacing its existing data structures. The original program already used appropriate structures for lookup and sorting, so I focused on weaknesses in how the data was validated.

I implemented and tested each change separately: duplicate detection, required-field validation, and prerequisite verification. I then ran regression tests to confirm that the original course loading, sorting, searching, and prerequisite display still worked.

One challenge involved creating valid CSV test data. An early test file was saved as an Excel workbook instead of plain-text CSV, which caused the loader to misread the file. Correcting the file format reinforced the importance of validating both the code and the data used to test it.

Overall, the enhanced version is stronger because it improves reliability without adding unnecessary complexity. It also reflects a more practical approach to software development by preserving what already works and making targeted improvements where they provide clear value.